SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5101

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

*I **N.AKSHAYA(192425129**) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: N.AKSHAYA

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB). Evaluate how secure hash functions improve integrity compared to basic checksum methods.

Aim:

To debug a SHA-based hashing program and improve its processing speed and data integrity.

Problem:

The program produces inconsistent hash outputs due to errors in message padding, block processing, bitwise operations, or endianness.

Algorithm:

1. Read the input file in binary mode.
2. Divide the file into small chunks.
3. Calculate the SHA-256 hash using Python.
4. Process large files chunk by chunk.
5. Display the final hash value.

Python Code:

```
import hashlib

with open("sample.txt", "w") as f:
    f.write("Hello Cryptography")

def calculate_hash(filename):
    sha = hashlib.sha256()
    with open(filename, "rb") as f:
        while True:
            data = f.read(1024 * 1024)
            if not data:
                break
            sha.update(data)
    return sha.hexdigest()

print("Hash:", calculate_hash("sample.txt"))
```

Output:

```
========== RESTART: C:/Users/aksha/Downloads/CSA5101-Cryptography/1.py =========
Hash: c1b2a7351baf40d991b3a1d88e2e82c9036b92df830fb82ed70850b8d820d3b5
>>
```

Analysis:

SHA-256 produces a fixed-length hash and helps detect changes in data. It is more suitable than basic checksums for detecting intentional manipulation. Processing files in chunks reduces memory usage and improves efficiency for files larger than 50 MB.

Result:

The program generates a consistent SHA-256 hash and processes large files efficiently.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption). Analyze the performance vs security trade-offs in TLS optimization.

Aim:

To debug SSL/TLS handshake failures and improve connection speed while maintaining security.

Problem:

Handshake failures may occur due to invalid certificates, incorrect hostname verification, or key exchange errors.

Algorithm:

1. Create a secure SSL context.
2. Enable certificate verification.
3. Connect to the server.
4. Establish a secure TLS connection.
5. Display the TLS version and cipher.

Python Code:

```
import ssl

import socket
```

```
def secure_connection(host):
    context = ssl.create_default_context()
    context.minimum_version = ssl.TLSVersion.TLSv1_2
    with socket.create_connection((host, 443)) as sock:
        with context.wrap_socket(
            sock, server_hostname=host
        ) as secure:
            print("TLS Version:", secure.version())
            print("Cipher:", secure.cipher()[0])
secure_connection("example.com")
```

Output:

```
========== RESTART: C:/Users/aksha/Downloads/CSA5101-Cryptography/2.py =========
TLS Version: TLSv1.3
Cipher: TLS_AES_256_GCM_SHA384
>
```

Analysis:

Certificate validation protects against fake servers. TLS session resumption can reduce handshake delays for repeated connections. However, disabling certificate verification or using outdated protocols can weaken security.

Result:

The program establishes a secure TLS connection and displays the negotiated version and cipher.

3. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption). Analyze the performance vs security trade-offs in TLS optimization.

Aim:

To correct digital signature verification errors and improve signature security.

Problem:

Invalid signatures may occur because of incorrect parameters, mismatched keys, or weak random number generation.

Algorithm:

1. Generate a DSA private key.
2. Obtain the public key.
3. Sign the message.
4. Verify the signature.
5. Display the verification result.

Python Code:

```
import hashlib

p = 61

q = 53

n = p * q
```

```
phi = (p - 1) * (q - 1)

e = 17

d = pow(e, -1, phi)

message = "Secure message"

h = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n

signature = pow(h, d, n)

print("Original Message:", message)

print("Digital Signature:", signature)

verified_hash = pow(signature, e, n)

if verified_hash == h:

    print("Signature is valid")

else:

    print("Signature is invalid")
```

Output:

```
========== RESTART: C:/Users/aksha/Downloads/CSA5101-Cryptography/3.py =========
Original Message: Secure message
Digital Signature: 1588
Signature is valid
>>>
```

Analysis:

Secure random numbers are essential for protecting private keys. Correct parameters and matching public and private keys help ensure successful verification. Cryptographic libraries reduce errors in mathematical operations.

Result:

The program generates and verifies a DSA digital signature successfully.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation